Git: un outil nécessaire pour les data scientists

Lino Galiana
Version du 20 octobre 2026

Une partie annexe au cours pour découvrir `Git`, un outil devenu indispensable pour les data scientists afin de mener des projets impliquant du code `Python`.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction **1**
1.1 Le problème 1
1.2 La solution technique: `Git` 2
2 Comment utiliser `Git` quand on fait du `Python` ? **2**
3 Contenu de la partie **3**
Informations additionnelles 3

Cette partie du site présente un élément qui n'est pas propre à `Python` mais qui est néanmoins indispensable : la pratique de `Git`.

Une grande partie du contenu de la partie provient d'un cours dédié fait avec Romain Avouac

Dérouler les *slides* ci-dessous ou cliquer ici pour afficher les slides en plein écran.

1 Introduction

1.1 Le problème

La démarche naturelle quand on commence à travailler longuement sur un projet pour lequel on veut éviter d'effacer et perdre son code à cause d'une erreur humaine est de dupliquer les fichiers et créer des **versions multiples d'un même code qui sont des instantanés à un moment donné** :

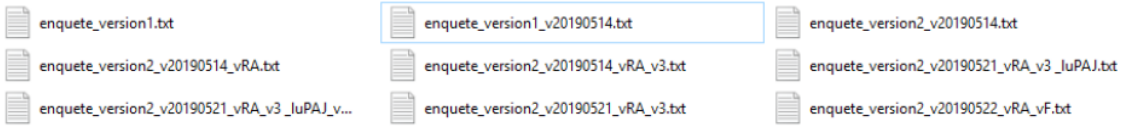


Figure 1. – Le contrôle de version artisanal

Ceci est une manière de sauvegarder l'évolution de son code et donc la vie d'un projet. Néanmoins c'est une manière très artisanale de procéder qui n'efface pas l'erreur humaine puisqu'il est toujours possible de faire une erreur lors de la duplication du fichier ou d'oublier de sauvegarder une version qui nous satisfaisait.

Cette pratique présente de nombreux inconvénients. Le premier est le fait que la recherche d'information pertinente, par exemple sur l'introduction d'un bug, est compliquée par cette structure dupliquée. Il est difficile, sans rentrer dans le détail de chaque fichier, de connaître son évolution réelle

entre deux dates. Pour l'extraction d'information, il serait beaucoup plus pertinent de construire des fichiers de l'évolution entre telle ou telle version mais cela requiert de nombreux gestes manuels et un temps conséquent. Le deuxième problème est qu'on ne sait pas forcément, quand on revient quelques jours plus tard sur le projet, pourquoi on a enregistré telle ou telle version : quand on est le 28 novembre, se souvient-on de ce qui distingue les versions du 4 mai 12:37 et 16:02 ?

Si on ajoute la dimension collaborative du travail sur le code, on voit rapidement d'autres inconvénients à cette approche artisanale. En premier lieu, quelqu'un reprenant le projet aura du mal à rentrer dans celui-ci. De plus, il sera encore plus difficile pour cette personne de rechercher l'information pertinente sur les choix de version faits: cet historique lui est finalement inutile. D'autant que se pose la question du partage de ces codes: par quel biais cet ensemble de fichiers est-il partagé ? Par mails ? Sur un disque partagé ? Mais que se passe-t-il si plusieurs participants du projet travaillent en même temps dessus ? Comment organiser la collaboration et réconcilier les changements s'ils ont lieu de manière conjointe ?

1.2 La solution technique: `Git`

`Git` apporte une réponse technique à ces très nombreuses questions. Ce logiciel spécialisé dans le contrôle de version, c'est-à-dire dans le suivi des évolutions d'un projet, vient résoudre de nombreux problèmes liés à la mise en oeuvre de projets de *data science* dans les organisations. L'objet de ce chapitre est de présenter quelques concepts nécessaires à la compréhension de `Git` et présenter la manière dont celui-ci vient aider à gérer l'évolution d'un projet informatique. Le chapitre suivant introduira à la manière dont `Git` fluidifie la collaboration au sein d'équipes impliquées dans des projets de *data science*. Ceci est devenu indispensable car l'époque où les *data scientists* travaillaient tous seuls sur des *notebooks* est révolue. Les projets de *data science* sont devenus, dans la plupart des organisations, plus ambitieux et impliquent souvent plusieurs personnes avec des profils diverses[1], il est donc nécessaire d'avoir de la discipline pour que la collaboration soit fluide.

2 Comment utiliser `Git` quand on fait du `Python` ?

`Git` est un logiciel de contrôle de version, c'est-à-dire un logiciel en charge d'enregistrer les évolutions d'un fichier dans le temps (ce qu'on appelle versionner). Ce n'est pas un *package* `Python`, ce ne sera pas par ce biais que nous l'utiliserons.

Les utilisateurs de `Python` peuvent utiliser `Git` par le biais de deux intermédiaires: par le biais de la ligne de commande ou par le biais d'extensions graphiques dans leurs environnements de développement (`VSCode`, `Jupyter`, etc.). Sur le `SSPCloud`, l'infrastructure recommandée pour ce cours, nous avons la possibilité pour faire du `Git` par les deux approches mais nous allons principalement utiliser l'interface graphique de `VSCode` qui réduit le coût d'entrée dans `Git`.

> Où trouver de l'information complémentaire à ce cours ?
>
> `Git` fait partie des pratiques collaboratives devenues standards dans le domaine de l'*open-source* mais également de plus en plus communes dans les administrations et entreprises de la *data science*.

[1] Ce constat est le point de départ du cours de « Mise en production de projets *data science* » disponible sur ensae-reproductibilite.github.io/website/ que Romain Avouac et moi donnons à la fin de la scolarité de l'ENSAE.

Il existe de nombreuses ressources sur l'utilisation de `Git` sur internet. Malheureusement, beaucoup sont techniques et font l'hypothèse d'une connaissance déjà poussée de certaines notions informatiques utiles à la compréhension de `Git`. Ce cours ne fera pas de telles hypothèses, à l'exception d'une connaissance minimale de la logique de *filesystem*, c'est-à-dire de la manière dont s'organisent des fichiers sur un ordinateur.

En ce qui concerne les contenus proches de celui-ci, une série de ressources de formation a été rassemblée par l'Insee sur ce site.

3 Contenu de la partie

L'apprentissage de `Git` est décliné en deux chapitres:

- Le premier est consacré à présenter la logique générale de `Git`, les concepts importants et à illustrer ceux-ci par la pratique dans un cadre de travail individuel.
- Le second chapitre est consacré aux enjeux liés au travail collaboratif

L'objectif général de cette partie est de démystifier `Git`. Si la découverte en autodidacte peut être particulièrement douloureuse, ce n'est pas le cas en étant accompagné par une ressource proposant une illustration des concepts importants par la pratique et complixifiant le cadre d'usage de `Git` de manière progressive.

> 👉 Un certain nombre de termes nouveaux quand on découvre `Git` mais qui constituent les concepts utiles à sa compréhension sont définis dans les marges des deux prochains chapitres comme cela est montré ici.

Informations additionnelles

i Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "⩾3.13,<3.14"
dependencies = [
    "altair⩾6.0.0",
    "cartiflette⩾0.2",
    "contextily==1.6.2",
    "duckdb⩾0.10.1",
    "folium⩾0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables⩾0.12.0",
```

```
    "gt-extras>=0.0.8",
    "ipykernel>=6.29.5",
    "jupyter>=1.1.1",
    "jupyter-cache>=1.0.0",
    "kaleido>=0.2.1",
    "langchain-community>=0.3.27",
    "loguru==0.7.3",
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.